

Workshop 3

Deploy a Trading Venue Component on Devnet

Read a real order book. Deploy a tiny one. Watch it settle.

Travis Sim

NUS · Singapore

Thu 21 Jan 2027 · COM3/COM4

Where we are

Tonight we move from agents to the venue they trade on.

DONE

Build night 3

Thu 12 Nov 2026

Position sizing and the trade ledger

TONIGHT

Workshop 3

Thu 21 Jan 2027

Deploy a venue component on devnet

NEXT

Build night 4

Thu 28 Jan 2027

Backtest validation and failure testing

Tonight

18:00 to 20:00. The headline is a program address and transaction links.

18:10 — 40 min

Slides

order book measures, on-chain or off-chain, sent or confirmed

18:50 — 25 min

Live demos

five demos, you predict first

19:15 — 15 min

Mini-lab

read one market, spread by hand, then in code

19:30 — 25 min

Open build

pick a starter, extend it, deploy

19:55 — 5 min

Push

one line on your next step

LEAVE WITH

A program address and transaction links

Goal and rules

Leave with a deployed program address and transaction links.

GOAL

One small venue component, running on devnet, that you can explain.

READS

Public mainnet data: order books, prices, swaps.

WRITES

Devnet only. Test tokens, a throwaway key, no real funds.

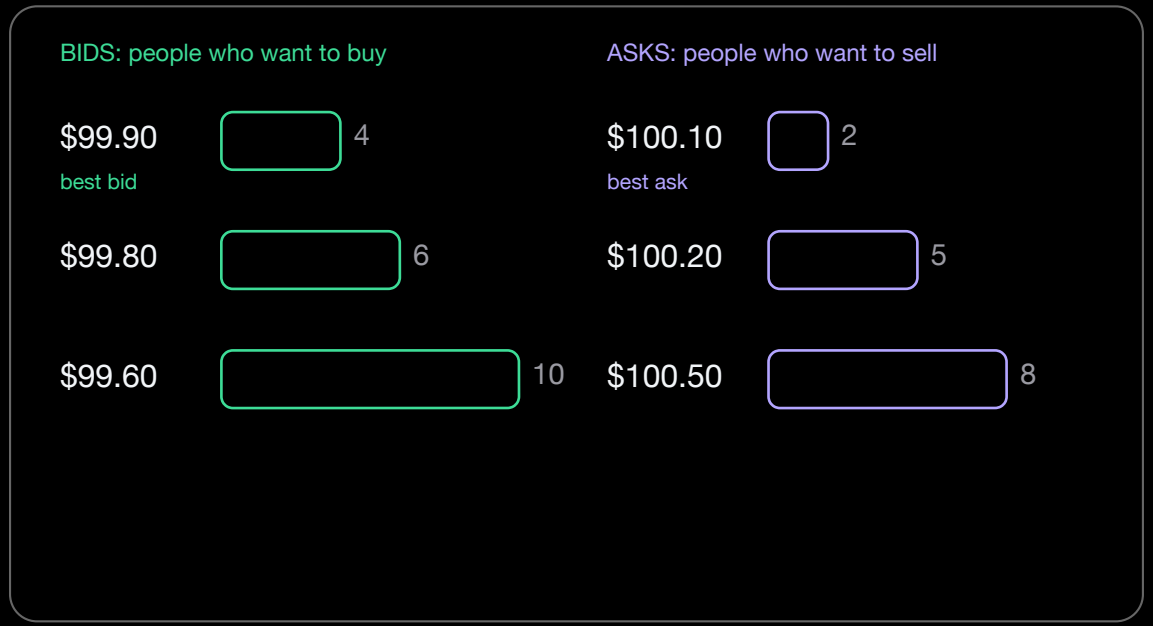
KEYS

Never paste a seed phrase. Keep key files out of git.

Reads use public mainnet data; every write is devnet only; no real funds.

An order book

Everyone's standing offers, sorted by price.



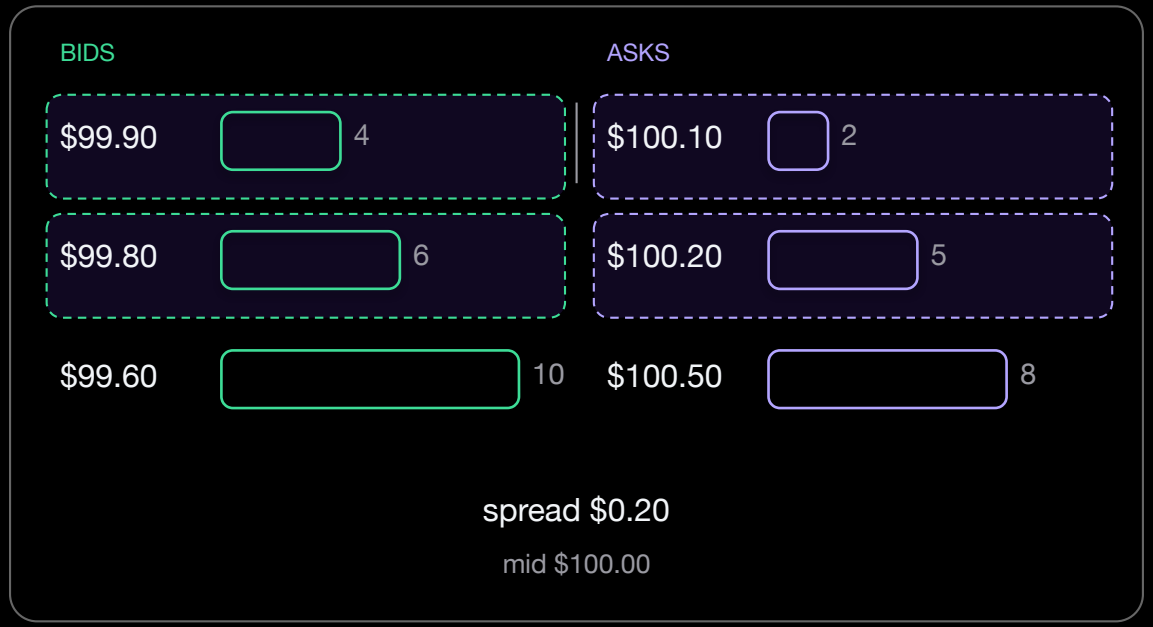
READ IT LIKE THIS

The best bid and the best ask are the top of the book. The gap between them is the spread.

Illustrative book. Price in dollars, size in units.

Spread and depth

How wide is the gap, and how much can you trade inside a band?



SPREAD

$0.20

20 bps of the mid

`spread = best ask − best bid`

DEPTH WITHIN 25 BPS

10 bid / 7 ask

size inside the dashed rows

`depth = size standing within a band of the mid`

A wide spread or thin depth means trading costs more.

Illustrative book, same as the last slide.

Imbalance and microprice

Which side is heavier, and where does the fair price lean?



IMBALANCE

+0.33

more size on the bid

```
imbalance = (bid size − ask size) /
(bid size + ask size)
```

MICROPRICE

$100.033

mid is $100.00: it leans up

```
microprice = size-weighted mid,
leaning to the thinner side
```

Positive imbalance means more buyers waiting than sellers.

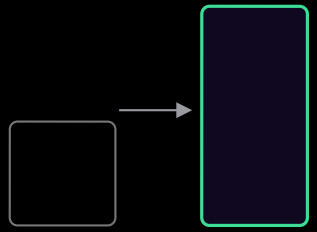
Illustrative book, top of book only.

Order flow imbalance

Not how the book looks, but how it changed.

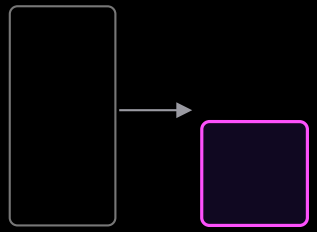
BUY PRESSURE

bid grows or ask shrinks



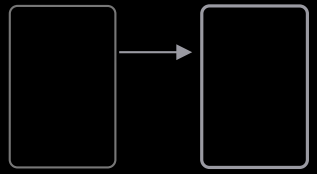
SELL PRESSURE

bid shrinks or ask grows



NOTHING

the top of the book did not move



Add up the changes between snapshots. A positive sum means buyers were adding.

Illustrative. Needs two snapshots of the book, so it needs a data collection run.

You can only compute it if you can see the book

This is why exchange builders matter.

Venue builders who publish their book make every measure here possible.

1

Spread

needs best bid and best ask

2

Depth

needs the whole ladder

3

Order flow

needs the book at two moments

Source: standard order book measures; the demos compute each one from a real book.

A real Phoenix ladder

SOL/USDC on mainnet, read straight from the market account.

BEST BID

$100.000

USDC per SOL

BEST ASK

$117.999

USDC per SOL

SPREAD

1,651 bps

ask minus bid, over the mid

ORCA, SAME MOMENT

$108.22

reference price

WHAT IS IN THE ACCOUNT

1,297 bids and 336 asks on the book

within 10 percent of the mid: 500 SOL bid, 311 SOL ask

Decoded by our own code, checked against the official SDK output.

Reads use public mainnet data; every write is devnet only; no real funds.

Source: code/out/ladder.json, recorded data captured 2026-10-21 19:49:58 UTC. Phoenix SOL/USDC market 4DoNfFBf..., public mainnet RPC.

Three markets, three spreads

Same decoder, same minute of data collection.

MARKET	CLUSTER	BEST BID	BEST ASK	SPREAD
SOL/USDC	mainnet	100.000	117.999	1,651 bps
JitoSOL/SOL	mainnet	1.2822	1.3100	215 bps
mSOL/SOL	mainnet	1.3878	2.2000	4,527 bps
SOL/USDC	devnet	none	175.705	one-sided
wSOL/USDC	devnet	none	none	empty

A spread of 1,000 bps is 10 percent. These legacy books are thin.

Source: code/out/ladder.json, first of 32 snapshots per market, captured 2026-10-21 19:49:58 UTC. Devnet markets use test tokens.

Thin books are the lesson

What the data collection run actually saw.

SOL/USDC, TOP FIVE LEVELS

changed 1 time

in 32 snapshots over about 8 minutes

A book that never moves carries no new information.

LESSON

A decoder that works is not the same as a market that is busy.

DEVNET MARKETS

SOL/USDC: no bids, 9 asks, 3 expired

wSOL/USDC: empty, 2 orders expired

TRY THIS

Always check how often the top of the book changes before you trust a feature.

Source: code/out/ladder.json, snapshots 15 s apart, captured 2026-10-21 19:49:58 UTC.

Flow without a book

Real Orca swaps, no order book needed.

TICK RULE

96%

2,633 of 2,750 swaps get the recorded side right

label each trade by the price move before it

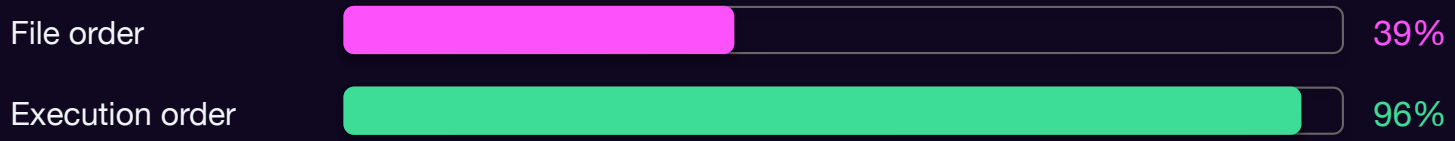
ROLL SPREAD

4.3 bps

estimated spread

spread estimated from how prices bounce

A DATA ORDER BUG, FOUND BY CHECKING



Inside one slot the recorded swaps were reversed. Put back in order, the tick rule works.

Source: code/out/flow.json, 2,900 recorded Orca SOL/USDC swaps.

On-chain or off-chain

Put on-chain only what must be trusted.

ON-CHAIN

- Who owns what
- The rule that moves funds
- The record of every fill

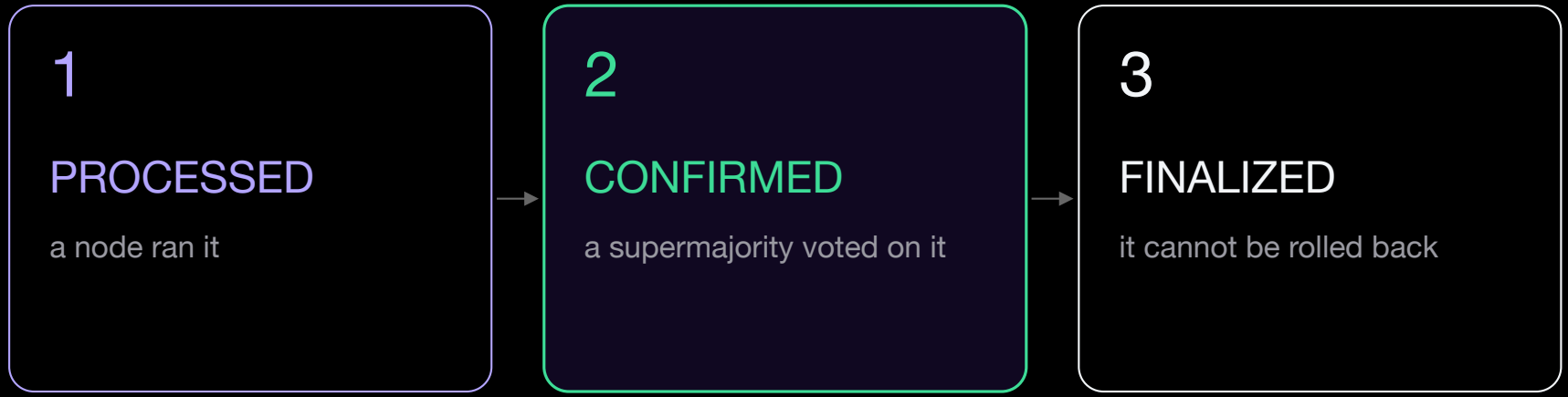
OFF-CHAIN

- Strategy and signals
- Charts and reports
- Retries and monitoring

Our order book component keeps the matching rule and the record on-chain. Everything around it stays off-chain.

Sent is not confirmed

A transaction passes through three commitment levels.



Each step is slower and safer. Credit money at the level your risk allows.

Illustrative.

Measured on devnet

Median time from sending to each commitment level, nine real transactions.

	PROCESSED	CONFIRMED	FINALIZED
DEVNET nine transactions, public devnet RPC	319 ms	712 ms	712 ms
LOCAL VALIDATOR same program, one machine	300 ms	331 ms	17.0 s

On this devnet, confirmed and finalized arrived together. A local validator waited about 17 seconds for finalized.

Source: code/out/devnet.json (devnet) and the local validator run, same program. Captured 2026-10-21 20:10:50 UTC.

The deposit watcher

Credit a deposit only when the transaction succeeded.

Credit at confirmed. Never credit a failed transaction.

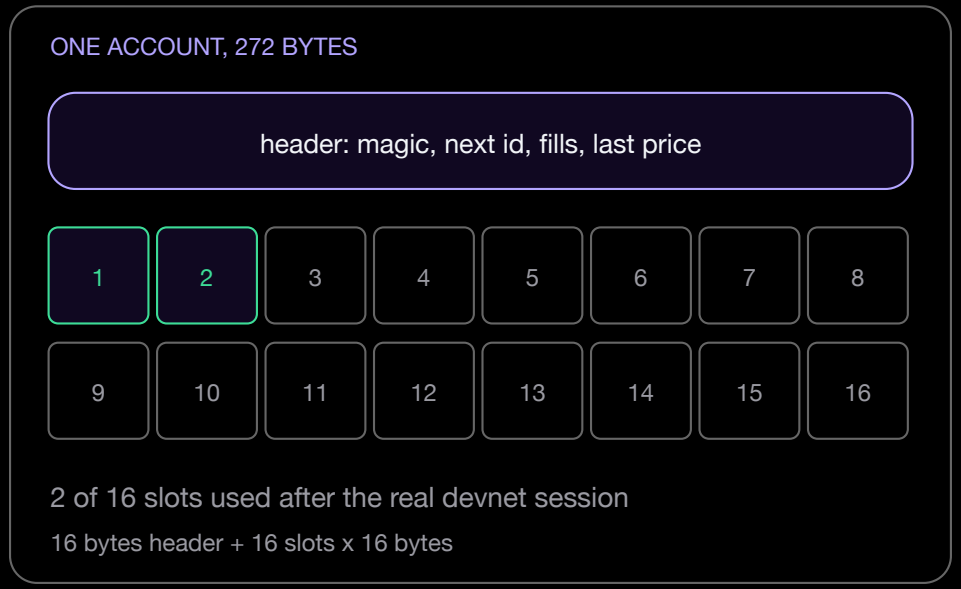
deposit 1	0.0010 SOL	confirmed in 932 ms	CREDITED
deposit 2	0.0015 SOL	confirmed in 534 ms	CREDITED
failing deposit	1.0660 SOL (more than held)	confirmed in 654 ms	NOT CREDITED

The failing one never reached the ledger: its transaction carried an error.

Source: code/out/devnet.json, devnet deposits to a fresh vault. Vault ARznxNAn..., captured 2026-10-21 20:01:14 UTC.

One small component: mini_book

A limit order book in a single account, 272 bytes.



Header

magic, next order id, fills, last price

16 slots

id, side, price, quantity

Instructions

init, place, cancel

Demo only: no balances, no owners, no signer checks. Not a venue.

Matching rules

Best price first, then the oldest order. Fill at the resting price.

1

Price priority

the best price fills first

2

Time priority

at one price the lowest order id fills first

3

Maker price

a fill happens at the resting order's price

4

Leftovers rest

unfilled quantity joins the book

REAL DEVNET LOG, STEP 7

```
ORDER id=6 ASK px=99 qty=9
FILL taker=6 maker=5 px=101 qty=2
FILL taker=6 maker=3 px=99 qty=6
REST id=6 qty=1
```

Example from the real devnet run: ask 9 at 99 meets bid 7 at 101 and bid 6 at 99.

Source: code/out/matching.json, step 7: ask 9 @ 99 crosses: price priority, then rests 1.

The matching loop

program/src/lib.rs (inside place)

```
let opp = 3 - side;
let mut left = qty;
while left > 0 {
    let m = best(b, opp, side, price);
    if m == SLOTS {
        break;
    }
    let o = H + m * 4;
    let (mid, mp, mq) = (r(b, o), r(b, o + 2), r(b, o + 3));
    let fill = if left < mq { left } else { mq };
    let mut l = Line::new();
    l.s("FILL taker=").n(id).s(" maker=").n(mid).s(" px=").n(mp).s(" qty=").n(fill);
    out.line(&l);
    left -= fill;
    if fill == mq {
        clear(b, o);
    } else {
        w(b, o + 3, mq - fill);
    }
    w(b, 2, r(b, 2).wrapping_add(1));
    w(b, 3, mp);
}
```

1 **Find the best**

The best resting order that crosses: price first, then oldest.

2 **Fill and log**

Fill the smaller side, write a FILL line, clear or reduce.

3 **Rest the rest**

No libraries, no allocator. Logs are the receipt.

Source: demo-repo/code/program/src/lib.rs (excerpt: indented one level, comments and blank lines removed)

Deployed on devnet

Real addresses. Open them in an explorer.

PROGRAM ID

`UQLHxoX7P4RV4sPV9mS4DFmXD1a6pi1qSkh2vjEWwrL`

DEPLOY TRANSACTION

`3vaz8GiD3DWejpUqorSYe51gBgnWonHdu8WTCFz8fNw44xJNzBoHVqHPemjnWchgd1RdYSE5GaZCH4ZxjBPgCChC`

PROGRAM FILE SHA-256

`866aef368c82ff3e47bb74ffa1098b4c3e218fb0f3bf6723922c1b01c131bcb2`

UPGRADE AUTHORITY AND PAYER

`2iGcpoF9xx5GFEXDJNpoY2VqB3B5kzimeufvriJCokYV`

SLOT

513,637,896

SIZE

5,528 bytes

ON-CHAIN = BUILD

yes, hashes equal

CLUSTER

devnet

solana program dump, then compare hashes

Funded by a transfer from a leftover devnet wallet, because the faucet returned 429 all day.

Source: code/out/devnet.json (deploy). Explorer: explorer.solana.com/address/<program id>?cluster=devnet

Nine real transactions

A scripted session against the deployed program.

#	WHAT HAPPENED	SIGNATURE	UNITS	FEE	CONFIRMED	RESULT
1	init: create the book account and initialize it	`5rm3EFXH...`	266	10,000	678 ms	ok
2	ask 5 @ 101 rests	`4WJ3Q2UY...`	1,237	5,000	711 ms	ok
3	ask 4 @ 102 rests	`4DxXEcyq...`	1,244	5,000	712 ms	ok
4	bid 6 @ 99 rests	`3FcCzUJo...`	1,248	5,000	788 ms	ok
5	bid 3 @ 98 rests	`36ed5hpQ...`	1,255	5,000	605 ms	ok
6	bid 7 @ 101 crosses, fills 5 @ 101, rests 2	`4n8dHBs2...`	2,156	5,000	739 ms	ok
7	ask 9 @ 99 crosses, price priority, then rests 1	`4Zb1G4nq...`	3,018	5,000	898 ms	ok
8	bid with quantity 0 is rejected (fee still paid)	`2QFwDfQB...`	141	5,000	696 ms	FAILED
9	cancel order 4	`3gRwfYsa...`	468	5,000	715 ms	ok

Fees in total: 50,000 lamports (0.00005 SOL) for 9 transactions, 1 failed.

Source: code/out/devnet.json. Click through from the demo or the explorer links. Captured 2026-10-21 20:10:50 UTC.

Rejected, and what this does not do

A failed transaction still costs a fee.

A REJECTED ORDER

custom error 5

Quantity 0 returned custom error 5. The book did not change, the fee was still charged.

```
fee charged: 5,000 lamports   units used: 141
```

WHAT THIS TOY DOES NOT DO

- No balances or collateral
- No owners or signer checks
- 16 orders, one book

Real venues add all three. That is why a venue is a team, not a weekend.

Source: code/out/devnet.json, step 8 of the real session.

Pick a starter

Each one reads real data and writes devnet only.

BEGINNER

Deposit watcher

ledger, commitment, failed transactions

```
$ npm run watch
```

STANDARD

Mini order book

new order type, more tests

```
$ npm run exercise
```

STANDARD

Phoenix ladder reader

spread, depth, imbalance, a feature for your agent

```
$ npm run ladder
```

ADVANCED

Orca whirlpool lab

price from sqrt price, swap reader

```
$ npm run orca
```

Source: demo-repo/code/package.json scripts (run from demo/code).

Three ways to take part

Devnet SOL is the one thing we cannot guarantee.

BEGINNER	No SOL	Read-only: Phoenix reader, Orca lab, recorded demos
STANDARD	About 0.003 SOL	Run the deposit watcher and place orders on our deployed book
ADVANCED	About 0.07 SOL	Deploy your own copy of the program

The faucet is rate limited. Leads will pre-fund throwaway keys. Never send funds from a real wallet.

Measured: the 9 transactions cost 50,000 lamports in fees; the book account needs 2,032,000 lamports of rent; the program file is 5,528 bytes.

Demo: order book ladder

PREDICT FIRST

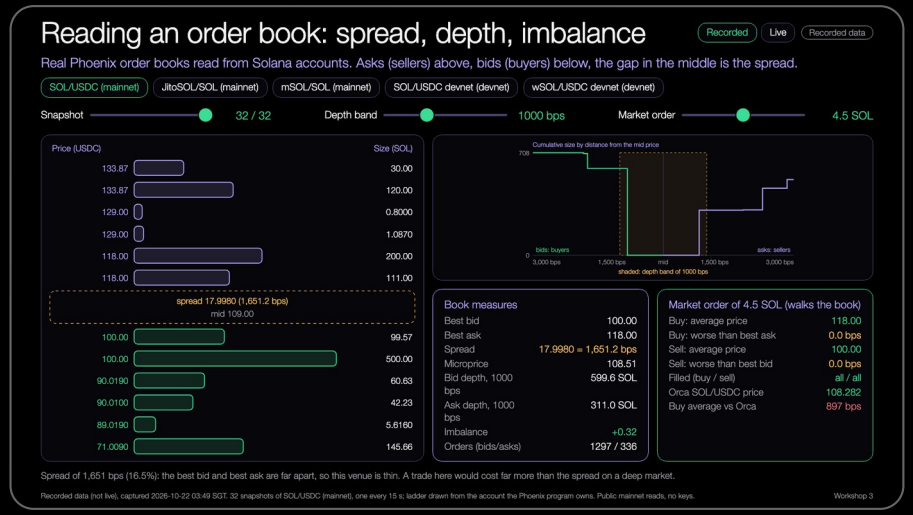
How wide is the SOL/USDC spread on this market, in basis points?

```
$ npm run ladder
```

YOU SHOULD SEE

- bids and asks as depth bars
- spread, depth bands, imbalance

Fallback: Recorded data is the default. No network needed.



Learning Lab demo, runs from disk, recorded data.

Try it: nus-solana-demos.vercel.app/0006/01-order-book-ladder.html

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data)

Demo: matching step by step

PREDICT FIRST

Which resting order fills first when an ask at 99 arrives?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- the book before and after each order
- the log lines the program writes

Fallback: Same recorded devnet run. The steps are the same every time.



Learning Lab demo, runs from disk, recorded data.

Try it: nus-solana-demos.vercel.app/0006/02-matching-step-by-step.html

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data)

Demo: devnet replay

PREDICT FIRST

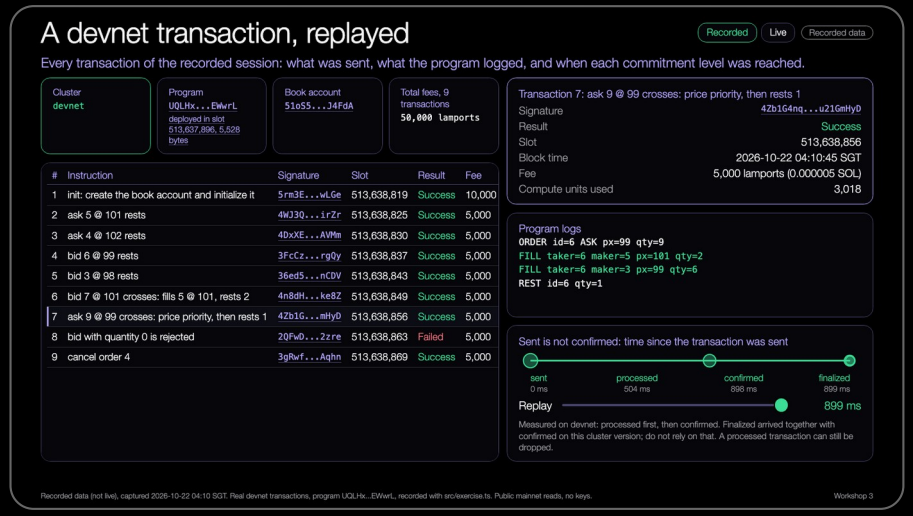
Which of the nine transactions fails, and does it still cost a fee?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- an explorer-style row per transaction
- processed, confirmed and finalized times

Fallback: Recorded devnet signatures. Nothing is sent.



Learning Lab demo, runs from disk, recorded data.

`Try it: nus-solana-demos.vercel.app/0006/03-devnet-replay.html`

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data)

Demo: deposit flow

PREDICT FIRST

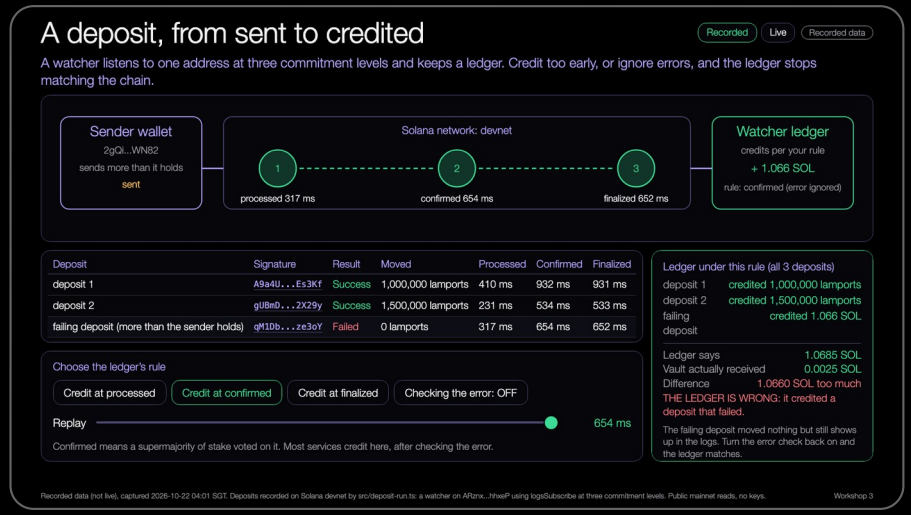
At which stage should a deposit be credited, and what if it failed?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- a wallet, a vault, a ledger
- sent and confirmed as two stages

Fallback: Recorded devnet deposits. Nothing is sent.



Learning Lab demo, runs from disk, recorded data.

Try it: nus-solana-demos.vercel.app/0006/04-deposit-flow.html

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data)

Demo: order flow imbalance

PREDICT FIRST

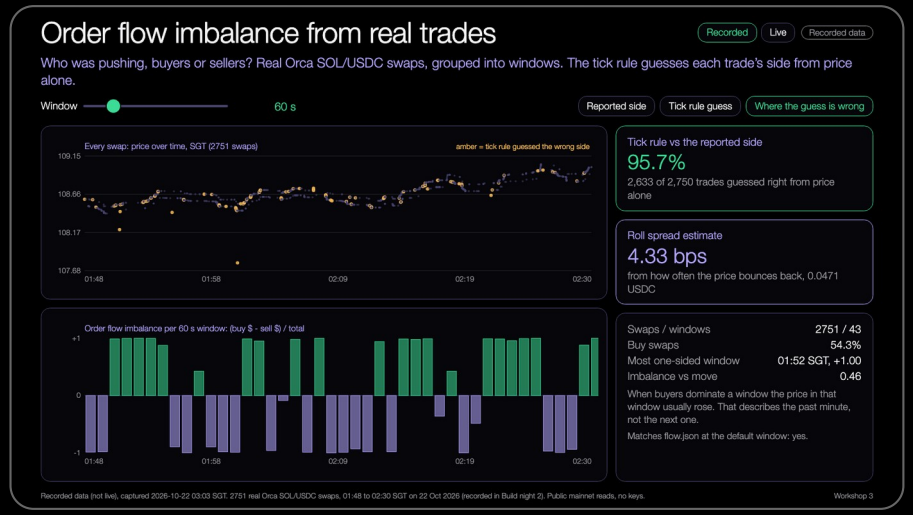
Does imbalance in one minute say anything about the next minute?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- one-minute windows of buys and sells
- the correlation with the same and next window

Fallback: Same recorded swaps. The windows are the same every run.



Learning Lab demo, runs from disk, recorded data.

`Try it: nus-solana-demos.vercel.app/0006/05-order-flow-imbalance.html`

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data)

Try the demos yourself

DEMOS

`nus-solana-demos.vercel.app/0006/`

Five demos. Recorded by default. Switch Live on to read real Solana data.

No install. The same files are in the repo and open from disk.



Scan to open the demos

Source: demo-repo/lab (the five Learning Lab demos, deployed as static pages, recorded data)

Mini-lab

Read one devnet market and compute the spread by hand, then in code.

1

Read

Open the ladder demo, pick a market

2

By hand

Best ask minus best bid, then divide by the mid

3

In code

npm run ladder, compare your number

4

Pick

Choose a starter and start extending it

Done when your hand number and your code number agree.

Takeaways

1. Spread, depth and imbalance all come from seeing the book.
2. Put on-chain only what must be trusted.
3. Sent is not confirmed. Credit at the level your risk allows.

DISCUSS

What does the on-chain component add for this user?

Does a successful transaction prove the project did the right thing?

Branch out

Where to take it next.

BEGINNER

Run the deposit watcher on a second kind of failure.

STANDARD

Add a new order type to mini_book, with a test.

ADVANCED

Add an owner and a signer check to the program.

Share back

Tell us what you built.

BY TONIGHT

1

Push your code and paste your program address.

WITHIN 48 HOURS

2

Post your transaction links and one lesson.

NEXT SESSION

3

Bring it to build night 4 for failure testing.

Addresses and signatures are public and safe to share. Never share a key or a seed phrase.

Get the code

REPO

`github.com/travisim/solana-agentic-trading-nus`

`folder: 0006-deploy-venue-component-devnet/demo`

```
1  $ cd demo/code && npm install
```

```
2  $ npm test
```

```
3  $ npm run demo -- --recorded
```

Recorded data is the default. Writes are devnet only.



Scan for the session folder

Let's connect



LinkedIn

linkedin.com/in/travisim



Telegram

t.me/travisim


SOLANA